

Why Merging LoRA Adapters Breaks Reasoning (But Not Fluency)

A worked explanation of the 17-notebook multi-task LoRA merging study on Qwen3-0.6B — what each experiment actually shows, why the numbers come out the way they do, and how the pieces connect into a single mechanistic story.

The original report is a dense results log: 17 notebooks, 9 merging algorithms, three benchmarks, and a lot of numbers with brief one-line inferences. This document restructures the same evidence into a causal chain — starting from a simple geometric fact about the three adapters, and following it through to why some merge methods destroy math reasoning while barely touching fluency or code. Each section explains the underlying concept first, then shows how the specific notebook results confirm or complicate it.

Roadmap

- **Part 1** — The setup: three specialists, one base model, nine ways to combine them.
- **Part 2** — The root geometric problem: magnitude imbalance and near-orthogonality.
- **Part 3** — Why SVD-style truncation survives this and element-trimming (TIES/DARE) doesn't.
- **Part 4** — Why SLERP fails for a different, subtler reason than the other methods.
- **Part 5** — What activation-space analysis (CKA) adds that weight-space analysis can't see.
- **Part 6** — The catalogue of fixes that were tried and why each one only half-worked.
- **Part 7** — The unifying explanation and the resulting design rules.

Part 1 — The Setup

The experiment starts with one base model, **Qwen3-0.6B**, and three independently trained LoRA adapters, each specialized on a different kind of data:

Adapter	Trained on	Skill it targets	Own benchmark
Dolly	Dolly-15k	General instruction-following / fluency	Perplexity (PPL)
MetaMath	MetaMathQA-15k	Multi-step arithmetic reasoning	GSM8K exact match
CodeAlpaca	CodeAlpaca-18k	Python code generation	HumanEval pass@1

Each adapter has rank $r=16$ and scale $\alpha=32$ (so the effective multiplier $\alpha/r = 2.0$), applied to all seven projection matrices in every transformer block (q, k, v, o, gate, up, down) — 196 individually-adapted weight matrices in total. A LoRA update to any single weight matrix is always a low-rank product: $\Delta W = B \cdot A$, where B is $d \times r$ and A is $r \times d$, so ΔW only ever has rank ≤ 16 no matter how large the base matrix W is.

Notebooks 1–3 just confirm that each specialist learned its intended skill (MetaMath's GSM8K EM jumps from the base model's near-zero to 0.22–0.53 depending on decoding settings; CodeAlpaca's HumanEval pass@1 reaches 0.17; Dolly's perplexity drops to 12.6). The interesting part is the side effect visible even here: CodeAlpaca's own Dolly perplexity is 38.4 versus Dolly's own 12.6 — training hard on code measurably wrecks conversational fluency in the same model. That's the first hint that these three skills pull the weights in substantially different directions, which is exactly what Notebook 4 goes on to measure directly.

The nine merge methods, in one line each

- **Linear** — plain average of the three ΔW 's. The naive baseline.
- **SVD (rank-16)** — add the three ΔW 's together (temporarily rank ≤ 48), then compress the sum back down to rank 16 with a truncated SVD. Sum first, compress second.
- **TIES** — drop the smallest 80% of parameters by magnitude, resolve sign conflicts by majority vote, average what survives.
- **DARE** — randomly drop 80% of parameters (independent of magnitude) and rescale the survivors by $1/(1-0.8) = 5\times$ to preserve the expected sum.
- **SLERP** — spherical (angle-preserving) interpolation between adapters, generalized pairwise across three adapters and tested across all 6 orderings since SLERP isn't associative.
- **BWSum** (the group's own method) — per-adapter rank-budget allocation combined via QR+SVD with a square-root weighting scheme, aiming to union the adapters' subspaces rather than just average their entries.
- **CT-calibrated merge** — rescale each ΔW to a common target norm before doing a plain linear average, directly targeting the magnitude imbalance found in Notebook 4.
- **Adaptive rank merge** — give each adapter a rank budget proportional to its own effective rank, truncate each ΔW *independently* to that budget, then sum.

- **Joint-SVD budget sweep** — the same sum-then-truncate recipe as plain SVD merge, but sweeping the final rank (16, 20, 24, 32, 40, 48) instead of fixing it at 16.

Two of these — adaptive rank merge and the budget sweep — exist specifically to isolate one question: does SVD merging work because of *how much* rank it keeps, or because of *when* it truncates (before vs. after summing the adapters)? That distinction turns out to be the whole story in Part 6.

Part 2 — The Root Problem: Magnitude Imbalance and Near-Orthogonality

2.1 — Magnitude imbalance (Notebook 4)

The Frobenius norm of a weight update, $\|\Delta W\|_F = \sqrt{\sum \Delta W_{ij}^2}$, is just a single number measuring "how much total change" an adapter made to a weight matrix, summed in quadrature across every layer. Notebook 4 computes this per-adapter, across all 196 layers:

Adapter	Total Frobenius norm	Relative to Dolly
Dolly	6.18	1.0×
MetaMathQA	15.40	2.5×
CodeAlpaca	30.74	5.0×

Why would code produce a 5× larger update than general chit-chat for the *same rank budget*? Because Dolly's singular values decay fast — most of its rank-16 capacity is barely used (this is exactly what the later effective-rank measurement in Notebook 13 confirms: Dolly only uses 77% of its rank-16 budget vs. ~89–90% for the other two). Code and math need to represent more distinct, high-precision transformations, so they spread their update energy across more of the available singular directions and end up with larger norms.

The consequence for a **plain linear average** is direct: if you add three vectors of very different length, the sum points mostly in the direction of the longest one. CodeAlpaca's update, being 5× larger than Dolly's, dominates the arithmetic mean — a naive merge is really "CodeAlpaca plus a small perturbation," not an equal blend of three skills. This single fact predicts, before any evaluation is even run, that linear merging should be biased toward code and against fluency/math — which is exactly the pattern later benchmarks show.

2.2 — Near-orthogonality (Notebook 9)

Magnitude imbalance is a problem you might expect to fix by rescaling — and that's exactly what the CT-calibrated merge (Notebook 7) and BWSum's weighting scheme try to do. But Notebook 9 uncovers a second, more fundamental problem that rescaling cannot touch: **direction**.

For every layer, the report computes the angle between each pair of adapters' updates using the standard cosine-similarity-to-angle conversion, $\theta = \arccos(\|\Delta W_A, \Delta W_B\| / (\|\Delta W_A\| \cdot \|\Delta W_B\|))$. A 90° angle means the two updates are orthogonal — they share no directional overlap at all, like two arrows at right angles. The measured means across all 196 layers:

- Dolly vs. MetaMath: 87.9° ($\sigma = 1.2^\circ$)
- Dolly vs. CodeAlpaca: 88.6° ($\sigma = 1.0^\circ$)
- MetaMath vs. CodeAlpaca: 88.9° ($\sigma = 0.9^\circ$)

Over 97% of all layers land within 5° of exactly perpendicular. This is not a coincidence specific to these three datasets — it's a generic property of high-dimensional spaces called **concentration of measure**. In a space with thousands of dimensions (a single weight matrix here has tens of thousands of entries), two

vectors produced by independent random-ish processes will almost always land close to perpendicular, simply because there are vastly more "sideways" directions than "aligned" ones to fall into. Three independently fine-tuned adapters, each pulling the same base weights toward a different task, behave like three independent random directions in a huge space — hence $\sim 88\text{--}89^\circ$ angles almost everywhere.

This is the paper's central mechanistic finding, and it's worth sitting with because it explains failures that look unrelated on the surface: **the three task updates are not "almost the same direction, scaled differently" — they are close to independent axes.** Any merge method that implicitly assumes the updates are similar in direction (which SLERP does, see Part 4) is standing on a false premise.

Part 3 — Why SVD Beats Element-Trimming (Notebooks 5, 6, 8, 10)

Given three near-orthogonal, differently-sized update vectors, how you combine them matters enormously. The report compares two philosophically different compression strategies:

3.1 — Two ways to shrink an update back to a fixed budget

- **Rank truncation (SVD/BWSum):** sum the three ΔW 's (temporary rank ≤ 48), take a thin SVD $\Delta W = U\Sigma V^T$, and keep only the top-16 singular directions (the ones carrying the most energy). This is a *structured* cut — you keep whichever coordinate axes matter most, wherever they point.
- **Element trimming (TIES/DARE):** look at the raw entries of the weight matrix and zero out 80% of them (smallest-magnitude for TIES, random for DARE), keeping the literal matrix entries rather than any notion of "principal directions." This is an *unstructured* cut.

The report quantifies the damage each approach does using **signal-energy retention**: for a singular-value spectrum $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_R$, the energy kept by retaining the top k directions is $\sum_{i=1}^k \sigma_i^2 / \sum_{i=1}^R \sigma_i^2$. Squaring the singular values matters because a matrix's total "information" (its squared Frobenius norm) is exactly the sum of its squared singular values — so this ratio is literally the fraction of the update's total energy preserved.

Notebook 10 computes this exactly, across all 196 layers:

Method	Kept	Energy retained	Energy lost
SVD rank-16 truncation	top 16 of 48 directions	94.93%	5.07%
TIES / DARE element trim	top 20% of raw entries	69.06%	30.94%

SVD keeps roughly 6× less "waste" than element trimming (5.07% lost vs. 30.94% lost) despite discarding a comparable fraction of raw capacity. Intuitively: singular value spectra are heavy-tailed, so a handful of top directions already capture almost all of a matrix's structure — cutting along those natural axes throws away mostly redundancy. Cutting individual matrix entries instead is directionally blind: it can just as easily hollow out a high-energy direction as a low-energy one, especially once you also apply TIES/DARE's sign-election and rescaling on top, which further distorts what remains.

3.2 — Where that shows up downstream (Notebook 8)

With bootstrap significance testing (10,000 resamples), SVD-merge's GSM8K score (0.155) is confirmed significantly better than Linear (0.11, $p=0.038$), and dramatically ahead of DARE (0.04, $p<0.001$). Math reasoning is fragile — GSM8K is a multi-step, low-tolerance task, so losing 31% of an adapter's encoded structure (as TIES/DARE do) is enough to collapse it almost entirely. But DARE simultaneously gets the *best* HumanEval score of all merge methods (0.238) — coding syntax, evaluated as pass/fail on generated code, tolerates a noisier but higher-variance parameter set better than reasoning does. This split — SVD wins reasoning, DARE wins code, no method wins fluency outright — is the first clear sign that different tasks have very different tolerances for structural damage, a theme that recurs through the whole report.

Part 4 — Why SLERP Fails, and Why Fixing the Obvious Bug Wasn't Enough

SLERP (spherical linear interpolation) is designed for a case where two vectors point in *related* directions and you want a smooth in-between vector that preserves their shared magnitude — the standard use case is interpolating between two similar model checkpoints. Part 2 already showed why that assumption fails here: the three adapters are ~88° apart, i.e. essentially orthogonal, not "related."

4.1 — What orthogonality does to SLERP specifically

SLERP's formula blends two unit vectors along the great-circle arc between them and then rescales by their magnitudes. When the angle between the inputs approaches 90°, the interpolation degenerates: the geometric machinery that's supposed to "rotate smoothly between two similar directions" instead behaves almost like a linear combination, but with the wrong norm-scaling law applied on top (SLERP's norm interpolation is itself nonlinear in the angle, and that nonlinearity is what breaks down at 90°). Empirically, GSM8K collapses to ≈0.01 regardless of the order the three adapters are merged in (all 6 permutations tested), which rules out associativity/ordering as the cause and points squarely at the orthogonality itself.

4.2 — Diagnosing exactly what breaks (Notebook 10, cross-referenced with later diagnosis)

A closer look at the failure found something specific: SLERP's linear norm interpolation over-inflates the magnitude of the merged update in the down_proj layers, in one case up to 157% of MetaMath's original scale. That is a genuine, fixable bug — the direction blending wasn't the problem, the magnitude scaling law was. The fix (Notebook 10's "SLERP norm fix") replaces linear norm interpolation with a **geometric mean** of the input norms, which is the norm-scaling law that actually matches how magnitudes should combine under angular interpolation.

4.3 — The fix worked... on exactly the thing it targeted

Checkpoint	Dolly PPL (↓ better)	GSM8K EM	HumanEval pass@1
Dolly specialist (reference)	12.63	0.025	0.165
SLERP, buggy norm	13.00	0.010	0.146
SLERP, norm-fixed	11.98	0.015	0.146

Fixing the norm law actually pushed Dolly perplexity *below* even the dedicated Dolly specialist (11.98 vs. 12.63) — spherical interpolation on near-orthogonal directions turns out to be an unusually good way to smooth the language-modeling surface. But GSM8K and HumanEval stay collapsed, essentially unchanged from the buggy version. The decoding-budget sweep later shows SLERP's math score can partially recover under much longer generation budgets (0.01 → 0.225 at 768 tokens), but only because SLERP-merged models ramble — they hit token caps 37.5% of the time (the report calls this the "chattiness tax") rather than reasoning more correctly per token.

This is the report's cleanest illustration of a general lesson: **a magnitude bug and a direction problem are separate failure modes, and fixing one doesn't fix the other.** The norm law only controls how long the

merged vector is; it has no way to rotate the vector's direction back into the task-specific subspace that GSM8K or HumanEval actually depend on. Perplexity is comparatively forgiving of small directional error as long as the magnitude is right; structured reasoning and code correctness are not.

Part 5 — What Activation Space Reveals (Notebooks 12–15)

Everything so far has been about the raw weight updates (ΔW). But a network's behavior is ultimately determined by its activations, not its weights directly — two different weight matrices can, in principle, produce very similar internal computations. The report's second analytical axis uses **Centered Kernel Alignment (CKA)**, a standard tool for measuring how similar two sets of neural activations are, invariant to rotation/scaling of the underlying representation.

5.1 — Why plain CKA doesn't work here, and the delta-CKA fix

Run naively, CKA between any two fine-tuned variants of the same base model saturates near 1.0, because the base model's own activations dominate the signal — the adapter-induced change is a small perturbation on top of a huge shared foundation. Notebook 12 fixes this by first subtracting the base model's activations from both networks, $\Delta h = h_{\text{model}}(x) - h_{\text{base}}(x)$, and only then computing CKA between the two *delta* activation sets. This isolates exactly the part of the representation that the adapter actually changed, the same design principle as subtracting a control condition in an experiment.

5.2 — The counter-intuitive correlation on math

For coding, the story is clean: HumanEval accuracy correlates strongly and positively with delta-CKA similarity to the CodeAlpaca specialist ($r = +0.80$) — the more a merged model's internal computation looks like the code specialist's, the better it codes, exactly as you'd expect.

Math reasoning breaks that pattern. Among TIES/DARE variants, GSM8K accuracy is *negatively* correlated with CKA similarity to the MetaMath specialist ($r = -0.895$) — and TIES/DARE actually have *higher* overall CKA scores (0.92, 0.92) than SVD (0.91) despite scoring far worse on GSM8K. Put plainly: TIES/DARE's merged model "looks more like" the math specialist in a coarse, whole-representation sense, yet cannot actually do the arithmetic. The explanation the report lands on is that GSM8K depends on a narrow, precise low-rank subspace within the representation being preserved intact — a small set of coordinates that carry the actual reasoning computation — while CKA's aggregate similarity score is dominated by the much larger bulk of the representation that's irrelevant to reasoning and easy to keep similar. Element trimming can preserve broad activation similarity while still severing the specific narrow pathway that reasoning runs through — coarse similarity and fine-grained functional preservation are different things, and TIES/DARE optimizes (accidentally) for the former.

5.3 — Where the damage concentrates: blocks 7–11, and attention over MLP

Two further measurements narrow down the location of this fragile subspace. First, **effective rank** — an entropy-based measure $\text{erank} = \exp(H(p))$ where $p_j = \sigma_j^2 / \sum \sigma_j^2$ — quantifies how many of an adapter's 16 available rank directions it actually uses in a non-trivial way. Dolly uses only 12.4 (77% of budget); MetaMath and CodeAlpaca use ~14.3 (89–90%). This matches Part 2's finding that Dolly's singular values decay fast: fluency needs fewer independent directions than reasoning or code do, which is also why giving Dolly the same rank budget as the other two (as every method here does) is somewhat wasteful for it.

Second, layer-by-layer delta-CKA shows representational divergence between merge methods is not uniform across the network's 28 blocks — it concentrates sharply in **blocks 7–11**, which the report identifies as intermediate "task-routing hubs." Hooking the attention and MLP sublayers separately within

that band shows attention projections consistently diverge more across merge methods than MLP layers do (mean CKA spread on GSM8K: attention 0.024 vs. MLP 0.017; on HumanEval: attention 0.033 vs. MLP 0.014). MLP layers, which mostly perform elementwise feature transformations, are comparatively robust to how you average or trim the weights feeding them. Attention projections, which route information between token positions based on precise learned directions, are far more sensitive to exactly which coordinates survive a merge. This is a specific, falsifiable localization of the abstract "orthogonal subspace" problem from Part 2 down to particular blocks and particular sublayers.

Part 6 — The Catalogue of Fixes, and Why Each One Only Half-Worked

With the diagnosis in hand (magnitude imbalance + near-orthogonal task subspaces + fragility concentrated in attention at blocks 7–11), the report tests four targeted interventions. Reading them side by side is the clearest way to see that this is fundamentally a *direction* problem that magnitude-only or budget-only fixes cannot reach.

6.1 — CT-calibrated merge: fixes magnitude, not direction

This method directly attacks the 5× norm imbalance from Part 2 by rescaling every adapter's update to a common target norm before linearly averaging: $\Delta W_i' = \Delta W_i \cdot (\|\Delta W\|_{\text{target}} / \|\Delta W_i\|_F)$. It works exactly as designed — Dolly perplexity improves to 12.95, nearly matching the dedicated Dolly specialist's 12.63, confirming that magnitude dominance really was suppressing fluency in the plain linear merge. But GSM8K drops to 0.06, worse than doing nothing (plain Linear's 0.11). Rebalancing the *lengths* of three near-orthogonal vectors doesn't change the fact that they're still pointing in different, largely non-overlapping directions — you've fixed the loudness problem while the message itself is still a wash of interference.

6.2 — SLERP norm-fix: already covered in Part 4

Same pattern: fixed Dolly PPL (which was a genuine magnitude bug), left GSM8K/HumanEval collapsed (which was never a magnitude problem to begin with).

6.3 — Independent adaptive rank merge: the clearest negative result

This method allocates each adapter its own rank budget in proportion to its measured effective rank (Dolly ≈ 7.2 , MetaMath ≈ 8.5 , CodeAlpaca ≈ 8.3 out of 16), truncates each adapter's ΔW to that budget *independently*, and only then sums the three truncated matrices. Across every total budget tested (16 through 48), GSM8K collapses completely to 0.0000 — worse than every other method in the entire study, including the buggy SLERP. The reason connects directly back to orthogonality: because the three adapters' top directions are close to perpendicular, truncating each one separately keeps only *that adapter's own* best directions and permanently discards everything else — including combinations that only become important once you look at the sum. Summing after independent truncation can't recover a shared low-rank structure because there mostly isn't one to recover; each adapter's discarded directions are gone before the merge even has a chance to consider them jointly.

6.4 — Joint-SVD budget sweep: rank-16 was already the right answer

This experiment isolates the opposite variable: keep the "sum-then-truncate" order that plain SVD merge uses (the thing adaptive rank merge got backwards), and instead sweep how much rank to keep after summing (16, 20, 24, 32, 40, 48). A small-sample proxy ($n=60$) suggested rank-20 might be better (EM=0.233), but the full $n=200$ evaluation reversed that: rank-20 scored 0.125 vs. rank-16's 0.155, and HumanEval likewise dropped (0.183 vs. 0.207). Extra rank past 16 adds directions that are mostly noise relative to the task-relevant subspace, and including them measurably hurts rather than helps. Combined with 6.3, this pins down that **the order of operations — sum first, then truncate — is the actual mechanism behind SVD's success**, not simply "how much capacity you allow." Summing first lets the

three near-orthogonal updates interact and lets the subsequent SVD find whatever shared structure exists across all of them; truncating first forecloses that interaction entirely.

The four fixes split cleanly into two categories: interventions on **magnitude** (CT-calibrated, SLERP norm-fix) each solved the specific magnitude symptom they targeted and left reasoning/code untouched; interventions on **rank-budget mechanics** (adaptive vs. joint truncation) revealed that truncation order, not budget size, is what determines whether shared structure survives. None of the four touch the underlying $\sim 88^\circ$ angle between task subspaces — which is why none of them close the reasoning gap.

Part 7 — The Unifying Explanation and the Resulting Design Rules

7.1 — Putting the causal chain together

- Three adapters trained independently on different task data end up with weight updates that are **near-orthogonal** ($\sim 88^\circ$) and **very different in magnitude** (up to 5×) — both are generic consequences of independent optimization in a very high-dimensional parameter space, not something specific to these three datasets.
- A merge method that just averages or interpolates these updates (Linear, SLERP) inherits both problems at once: the largest-magnitude adapter dominates, and interpolating between near-perpendicular vectors is mathematically closer to an incoherent blend than a meaningful compromise.
- Compressing back to a fixed budget by **trimming individual weight entries** (TIES, DARE) can look successful on coarse, high-tolerance metrics (perplexity, pass/fail code correctness, whole-network CKA) while quietly destroying a narrow, high-precision subspace that only structured multi-step reasoning (GSM8K) actually needs — because that subspace is a small fraction of the parameters/activations by volume, even though it's functionally critical.
- Compressing by **rank truncation after summing** (SVD/BWSum) is structurally the right move: singular value spectra are heavy-tailed, so keeping the top directions of the *combined* update preserves 94.93% of total energy and — because the truncation happens after the three (near-orthogonal, so largely non-overlapping) updates have already been summed — it's the only method given a chance to find genuinely shared structure across all three tasks.
- The fragility is not uniform across the network: it concentrates in **attention projections in blocks 7–11**, which act as intermediate task-routing hubs, while MLP layers and early/late blocks are far more tolerant of how the merge is done.
- Fixes aimed at magnitude (CT-calibration, SLERP's norm law) reliably fix magnitude-caused symptoms (fluency/perplexity) and reliably fail to touch direction-caused symptoms (reasoning/code) — the two problems are independent and require independent remedies.

7.2 — Practical design rules that follow directly from this evidence

- **Always truncate jointly.** Sum the adapter updates first, then run SVD on the combined matrix. Never truncate each adapter to a rank budget before summing — Notebook 16 shows this collapses reasoning entirely, regardless of how the per-adapter budget is chosen.
- **Default to SVD-style merging when reasoning tasks matter** — it retains 94.93% of update energy vs. 69.06% for TIES/DARE, and this gap tracks almost exactly onto the GSM8K gap between methods.
- **Use DARE (or similar element-drop methods) when the target skill is more forgiving of internal structure**, such as pass/fail code generation, where DARE outperformed every other merge method in this study.
- **Calibrate magnitude first if fluency/perplexity is a priority**, but treat this as necessary and not sufficient — it will not by itself protect reasoning or code accuracy.

- **Avoid SLERP for three or more independently-trained adapters.** High-dimensional concentration of measure makes near-orthogonality the default outcome, not an edge case, so SLERP's core assumption (related, similarly-directed inputs) is violated by construction. It remains reasonable for interpolating between two closely related checkpoints (e.g. two checkpoints of the same fine-tune at different steps).
- **If further gains are sought, look at blocks 7–11's attention projections specifically** — this is where the report's own evidence says the actual damage is concentrated, making it the natural target for any future block-adaptive or attention-specific merging strategy.

Read as a whole, the 17 notebooks build one continuous argument rather than 17 separate experiments: a geometric fact about independently-trained adapters (orthogonality + magnitude imbalance) predicts which merge methods should struggle; weight-space energy accounting explains why SVD-style truncation is the structurally sound way to compress under that geometry; activation-space CKA analysis shows the resulting damage is concentrated in a narrow, functionally critical subspace rather than spread evenly, and pinpoints where in the network that subspace lives; and every attempted fix confirms the diagnosis by successfully treating the symptom it targeted while leaving every other symptom untouched.